Mesos Model Class Diagram - Current State

Factory
ParserJson
DefaultGameFactory
-nextGameId : AtomicInteger
+createGame(players : ArrayList<Player>) : Game
GameFactory
+createGame(players : ArrayList<Player>) : Game
creates
creates
creates
uses
status
currentEra
status
GameState
ActionResult
-turnState : TurnState
-expectedPlayer : String
-turnEnded : boolean
-roundEnded : boolean
-gameEnded : boolean
TurnOrderTrack
-turnOrder : List<Player>
-currentPlayerIndex : int
-roundNumber : int
+setNewOrder(newOrder : List<Player>) : void
+getCurrentPlayer() : Player
+getRoundNumber() : int
+advanceToNextPlayer() : void
+isLastPlayer() : boolean
Game
-id : int
-players : ArrayList<Player>
-currentEra : Era
-status : GameStatus
-tribeDeck : TribeDeck
-buildingDeck : BuildingDeck
-board : Board
-turnOrderTrack : TurnOrderTrack
-turnState : TurnState
+setCurrentEra(currentEra : Era) : void
+setStatus(newstatus : GameStatus) : void
+getPlayers() : ArrayList<Player>
+getId() : int
+getCurrentEra() : Era
+getStatus() : GameStatus
+setup() : void
+moveTotem(player : Player, offerSpaceIndex : int) : void
+chooseCard(player : Player) : void
+calculateFinalScore() : void
snapshot
turn order
TurnState
turn state
Orchestration
GameOrchestrator
-game : Game
owns
start match
bootstraps games
lobby
LobbyManager
-nextLobbyId : int
-lobbies : Map<Integer, Lobby>
-gameFactory : GameFactory
+createLobby(host : Player) : Lobby
+joinLobby(lobbyId : int, player : Player) : void
+exitLobby(lobbyId : int, player : Player) : void
+removePlayer(lobbyId : int, player : Player) : void
+canStart(lobbyId : int) : boolean
+listOpenLobbies() : List<Lobby>
+getLobby(lobbyId : int) : Optional<Lobby>
+createGameFromLobby(lobbyId : int) : GameOrchestrator
managed lobbies
*
Lobby
MIN_PLAYERS : int
MAX_PLAYERS : int
-id : int
-hostNickname : String
-players : List<Player>
-status : LobbyStatus
+getId() : int
+getHostNickname() : String
+getMinPlayers() : int
+getMaxPlayers() : int
+getStatus() : LobbyStatus
+addPlayer(player : Player) : void
+removePlayer(player : Player) : void
+isHost(player : Player) : boolean
+transferHostToSecondPlayer() : void
+canStart() : boolean
+isFull() : boolean
+getPlayers() : List<Player>
board
enums
Era
first
second
third
RoundPhase
PLAYER_TURN
LOWER_ROW_EVENTS
RETURN_TO_STARTING_TILE
STARTING_TILE_EFFECTS
ROW_CLEANUP
ROW_REFILL
ERA_ADVANCEMENT
GameStatus
SETUP
IN_PROGRESS
ENDED
LobbyStatus
WAITING
READY
STARTED
CLOSED
InventionType
BOW
FIRE
WHEEL
KNIFE
RAFT
BASKET
SPEAR
TENT
NECKLACE
PAINTING
CharacterType
HUNTER
SHAMAN
ARTIST
BUILDER
INVENTOR
GATHERER
TotemColor
RED
BLUE
WHITE
YELLOW
BLACK
BenefitType
SETFORFOOD
PRESTIGEPOINTSFORSET
SUSTENANCEDISCOUNTARTISTS
SUSTENANCEDISCOUNTGATHERER
EXCLUDELOSSFROMSHAMANEVENT
BONUSFORTOTEMPOSITION
SHAMANMULTIPLIER
PAIROFINVENTORWITHSAMEINVENTION
BONUSSTARSDURINGSHAMANEVENT
SUSTENANCEDISCOUNTINVENTOR
BONUSHUNTEREVENT
cards
buildings
SetForFood
-rewardFood : int
-characterTypeCountMap : Map<CharacterType, Integer>
+addCharacterType(type : CharacterType) : void
+hasBonus() : boolean
+decrementCharacterType(type : CharacterType) : void
+decrementAllCharacterTypes() : void
+getCharacterTypeCount(type : CharacterType) : int
+getRewardFood() : int
tribeDeck
buildingDeck
setup/refill
setup/refill
TribeDeck
BuildingDeck
Deck
-cards : ArrayList<Card>
+draw() : Card
+isEmpty() : boolean
+shuffle() : void
cards
characters
Inventor
-inventionType : InventionType
+getInventionType() : InventionType
totem
Shaman
-stars : int
+getStars() : int
Hunter
-hasEffect : boolean
+getHasEffect() : boolean
Gatherer
Builder
-priceDiscount : int
-prestigePoints : int
+getPriceDiscount() : int
+getPrestigePoints() : int
Artist
CharacterCard
+isDrawable(player : Player) : boolean
players
resolveAction()
events
SustenanceEventCard
+applyEffect(players : ArrayList<Player>) : void
ShamanRitualEventCard
-prestigePointsForHigher : int
-prestigePointsForLower : int
+applyEffect(players : ArrayList<Player>) : void
HuntEventCard
-foodReward : int
-prestigePointsReward : int
+applyEffect(players : ArrayList<Player>) : void
characters
CavePaintingEventCard
-limitForPositive : int
-prestigePointsForPositive : int
-prestigePointsForNegative : int
+applyEffect(players : ArrayList<Player>) : void
EventCard
+isDrawable(player : Player) : boolean
EventAction
+applyEffect(players : ArrayList<Player>) : void
resolves last
players
resolves
BuildingCard
-foodCost : int
-prestigePoints : int
-hasEffectDuringMatch : boolean
+isDrawable(player : Player) : boolean
Card
-id : int
-era : Era
+getId() : int
+getEra() : Era
+setEra(era : Era) : void
CardAction
+getEra() : Era
+isDrawable(player : Player) : boolean
TribeCard
building cards
buildings
remove
tribe cards
player
Player
-nickname : String
-food : int
-prestigePoints : int
-totem : TotemColor
-tribe : Tribe
+getNickname() : String
+gainFood(amount : int) : void
+payFood(amount : int) : void
+gainPrestigePoints(amount : int) : void
+payPrestigePoints(amount : int) : void
+getFood() : int
+getTribe() : Tribe
2..5
tribe
Tribe
-characters : ArrayList<CharacterCard>
-buildings : ArrayList<BuildingCard>
+addCharacter(c : CharacterCard) : void
+addBuilding(b : BuildingCard) : void
+countHunters() : int
+countArtists() : int
+countGatherers() : int
+getShamansStars() : int
+getCharactersSize() : int
+hasBuildingForShamanRitual() : boolean
board
Board
-upperPile : UpperPile
-lowerPile : LowerPile
-offerPath : OfferPath
+buildOfferPath() : void
+setupInitialRows(tribeDeck : TribeDeck, buildingDeck : BuildingDeck, playerCount : int) : void
+shiftUpperToLower() : void
+refillUpperTribeCards(tribeDeck : TribeDeck, playerCount : int) : void
+moveUpperBuildingCardsToLowerRow() : void
+refillUpperBuildingCards(buildingDeck : BuildingDeck) : void
+removeUpperCard(card : Card) : void
+removeLowerCard(card : Card) : void
lower
upper
offer path
LowerPile
-tribeCards : List<TribeCard>
-buildingCards : List<BuildingCard>
+addTribeCards(cards : List<TribeCard>) : void
+addBuildingCards(cards : List<BuildingCard>) : void
+getAllCards() : List<Card>
+discardLowerRow() : void
+discardLowerBuildings() : void
+resolveLowerRowEvents(players : ArrayList<Player>) : void
+removeCard(card : Card) : void
event targets
occupiedBy
occupiedBy
offerpath
order extraction
OfferPath
-startingTile : StartingTile
-offerSpaces : List<OfferSpace>
+clearOfferSpaces() : void
+getPlayersInOrder() : List<Player>
+returnPlayersToStartingTile() : void
offer spaces
OfferSpace
-positionIndex : int
-code : String
-minPlayers : int
-foodBonus : int
-topRowCardsToTake : int
-bottomRowCardsToTake : int
-occupiedBy : Player
+occupy(player : Player) : void
+clear() : void
+getOccupiedBy() : Player
+isEnabledFor(playerCount : int) : boolean
+isAvailable() : boolean
+resolveAction(game : Game, player : Player) : void
+getPositionIndex() : int
+getCode() : String
place/effects
StartingBlock
-position : int
-foodModifier : int
-prestigeModifier : int
-occupiedBy : Player
+occupy(player : Player) : void
+clear() : void
+getOccupiedBy() : Player
+getFoodModifier() : int
+getPrestigeModifier() : int
+getPosition() : int
StartingTile
-startingBlocks : List<StartingBlock>
+initializeBlocks(playerCount : int) : void
+placePlayers(orderedPlayers : List<Player>) : void
+resolveEffects() : void
starting tile
blocks
UpperPile
-tribeCards : List<TribeCard>
-buildingCards : List<BuildingCard>
+addTribeCards(cards : List<TribeCard>) : void
+addBuildingCards(cards : List<BuildingCard>) : void
+getAllCards() : List<Card>
+removeAndReturnAllTribeCards() : List<TribeCard>
+removeAndReturnAllBuildingCards() : List<BuildingCard>
+removeCard(card : Card) : void
building cards
tribe cards
ranking
«record»
RankingResult
+playerRank : int
+leaderboard : List<LeaderboardEntry>
output
leaderboard
RankingService
+registerUser(nickname : String) : boolean
+isNicknameAvailable(nickname : String) : boolean
+recordMatchAndGetRanking(gameId : String, playerCount : int, playedAtUtc : Instant, requestingNickname : String, finalScores : List<MatchScore>) : RankingResult
+getLeaderboard() : List<LeaderboardEntry>
+getMatchHistory() : List<MatchHistory>
+getMatchHistory(playerCount : int) : List<MatchHistory>
«record»
LeaderboardEntry
+nickname : String
+elo : int
+wins : int
+winStreak : int
+rank : int
«record»
RankingConfig
+minPlayers : int
+maxPlayers : int
+initialElo : int
+minElo : int
+baseK : int
+kStepPerExtraPlayer : int
+streakBonusStep : int
+streakBonusCap : int
+maxBusyRetries : int
defaultConfig() : RankingConfig
current ratings
«record»
RatingSnapshot
+nickname : String
+elo : int
+gamesPlayed : int
+wins : int
+winStreak : int
SqliteRankingService
-config : RankingConfig
-database : SqliteDatabaseSupport
-validator : RankingInputValidator
-eloCalculator : EloRatingCalculator
+registerUser(nickname : String) : boolean
+isNicknameAvailable(nickname : String) : boolean
+recordMatchAndGetRanking(gameId : String, playerCount : int, playedAtUtc : Instant, requestingNickname : String, finalScores : List<MatchScore>) : RankingResult
+getLeaderboard() : List<LeaderboardEntry>
+getMatchHistory() : List<MatchHistory>
+getMatchHistory(playerCount : int) : List<MatchHistory>
config
leaderboard rows
pre-match state
input
EloRatingCalculator
-minPlayers : int
-baseK : int
-kStepPerExtraPlayer : int
-streakBonusStep : int
-streakBonusCap : int
-minElo : int
elo engine
placements
«record»
MatchScore
+nickname : String
+finalScore : int
computed updates
persistence batch
history mapping
validator
history rows
«record»
PlayerMatchUpdate
+nickname : String
+finalScore : int
+placement : int
+eloBefore : int
+eloDelta : int
+eloAfter : int
+gamesAfter : int
+winsAfter : int
+winStreakAfter : int
RankingInputValidator
-minPlayers : int
-maxPlayers : int
normalized scores
«record»
MatchHistory
+gameId : String
+playedAtUtc : Instant
+playerCount : int
+ranking : List<MatchRankingEntry>
ranking
«record»
MatchRankingEntry
+nickname : String
+finalScore : int
+placement : int
+eloBefore : int
+eloDelta : int
+eloAfter : int
+winStreakAfter : int
SqliteDatabaseSupport
-jdbcUrl : String
-maxBusyRetries : int
db support
schema/sql
SqliteRankingQueries
«exception»
DuplicateGameException
«exception»
RankingDataException
IllegalStateException

Placeholder class.
JSON deck parsing is not implemented yet.

Placeholder for the turn state machine.
No fields or behavior are implemented yet.

Core aggregate already exists, but setup,
move, draft and final-score flows are still empty.

Board owns the runtime rows and offer path.
Several setup/refill methods are still TODO.

Ranking is the most complete subdomain:
validation, Elo computation, persistence and queries
are already wired together via SQLite.

• = composition / owned part
o= aggregation / grouped element
dashed arrow = dependency / usage
notes = current implementation gaps or TODOs already present in code